Consuming the E-Commerce Stream into a Live Grafana Dashboard

Assignment 2 left four Kafka topics filling up with clickstream events, CDC order changes, payment webhooks and inventory rows. This stage closes the loop from the Assignment 1 flowchart. A single consumer reads all four topics, cleans each record, and writes it into MySQL. Grafana reads MySQL every five seconds, so the dashboard moves while the producer is still sending. The last section covers what the consumed data says about the business and what a manager should do about it.

1. Consumer Design (consumer.py)

The consumer subscribes to `clickstream-topic`, `orders-topic`, `payments-topic` and `inventory-topic` under one consumer group, `ecom-dashboard-consumer`. Every message is routed by topic name to a small cleaning function, and each function returns the rows it wants written. There are no classes. A dictionary maps topic to function, which keeps the main loop at about fifteen lines.

Topic	MySQL table	Cleaning applied
clickstream-topic	clickstream_events	Null user_id becomes GUEST with an is_guest flag. Event type and search term lower-cased. ISO timestamp parsed to UTC datetime. IP address dropped.
orders-topic	orders	Debezium envelope unwrapped. The after image is kept, or the before image for a delete (status set to DELETED). Status and payment method upper-cased. One row per order_id holds its latest state.
payments-topic	payments	Status and method upper-cased, currency defaulted to INR. Card last four digits and UPI VPA dropped before storage.
inventory-topic	inventory, products	Negative stock clipped to zero. Product name, category and price split into a products dimension so orders can be grouped by category.
all topics	stream_log	One row per message with topic, partition, offset and time consumed. Feeds the live throughput panel.

Every write is an upsert on the natural key, so replaying a topic from the start never creates duplicate rows. This matters because the consumer uses manual offset commits. The offset is committed only after MySQL has committed the row, which means a crash between the two steps replays the message instead of losing it. The upsert makes that replay harmless.

```
def upsert(table, row):
    cols = ", ".join(row)
    marks = ", ".join(["%s"] * len(row))
    updates = ", ".join(f"{c} = VALUES({c})" for c in row)
    sql = f"INSERT INTO {table} ({cols}) VALUES ({marks}) ON DUPLICATE KEY UPDATE {updates}"
    return sql, tuple(row.values())

for msg in consumer:
    rows = CLEANERS[msg.topic](msg.value)
    for table, row in rows:
        cur.execute(*upsert(table, row))
    cur.execute("INSERT INTO stream_log (topic, kafka_partition, kafka_offset) VALUES (%s, %s,
%s)",
                (msg.topic, msg.partition, msg.offset))
    db.commit()
    consumer.commit()        # offset moves only after MySQL has the row
```

A record that fails cleaning (a missing key or a value that will not parse) is rolled back, counted as rejected and skipped, so one bad message cannot stop the stream. The keying decision from Assignment 2 pays off here. Because orders are keyed by order_id, the create and update events for one order sit on the same partition and arrive in order, so the consumer can overwrite the order row without checking timestamps. The terminal prints one line per message and a per-topic total on exit:

```
[0003] orders-topic       p0 off=0    ORD50085  PLACED     INR   6,174.05
[0004] inventory-topic    p0 off=2    SKU-ELEC-102  WH-EAST-04  available=133
[0005] orders-topic       p0 off=1    ORD50093  CANCELLED  INR   1,358.30
[0006] payments-topic     p0 off=0    ORD50002  PayU      REFUNDED  INR  13,883.69
[0011] clickstream-topic  p0 off=0    U1015  add_to_cart      P103
...
[0400] clickstream-topic  p0 off=99   U1031  page_view        P106
consumed: clickstream-topic=100, orders-topic=100, payments-topic=100, inventory-topic=100 |
rejected=0
```

2. Live Dashboard

The stack runs from one `docker-compose.yml`: a single-node Kafka broker, MySQL 8 and Grafana 11. The MySQL data source and the dashboard are provisioned from files, so Grafana opens straight onto the dashboard with no manual setup. The dashboard auto-refreshes every five seconds. With the producer sending five messages a second, the KPI tiles and charts visibly change between refreshes.

Panel	Type	Question it answers
KPI row (6 tiles)	Stat	Net revenue, order count, average order value, payment success rate, messages consumed, low-stock positions
Live stream throughput	Time series	Is data still flowing, and from which topic? Messages per 10 seconds, stacked by topic
Net revenue by category	Bar chart	Which categories carry the revenue?
Clickstream funnel	Horizontal bar	Where do shoppers drop between page view and purchase click?
Payment outcome by gateway	Stacked bar	Is any gateway failing more than the others?
Order status mix	Donut	How much of the order book is still unfulfilled or cancelled?
Net revenue by shipping city	Bar chart	Which cities are worth prioritising for delivery and marketing?
Low-stock alerts	Table	Which product and warehouse pairs are below reorder level, and by how much?



Figure 1: Grafana dashboard refreshing every 5 seconds while the producer streams into Kafka.

3. Business Insights

All figures below come from the full run of 400 messages, 100 per topic. Once cancelled orders are excluded, the stream carried the following headline numbers.

₹9.44 L	100	₹10,155	75%	7	11 of 37
Net revenue	Orders	Avg order value	Payment success	Orders cancelled	Stock positions low

3.1 One phone carries almost half the revenue

The Smartphone 128GB made ₹4.05 lakh from 11 orders, which is 43 percent of net revenue. Mobiles and Home together account for 78 percent. The Bluetooth Speaker sold the most units of any product (39) but brought in only 9 percent of revenue. Grocery and Beauty together are under 2 percent. Revenue is concentrated in a few high-ticket SKUs, so a stock-out or a price war on one phone model hits the top line far harder than trouble anywhere else in the catalogue.

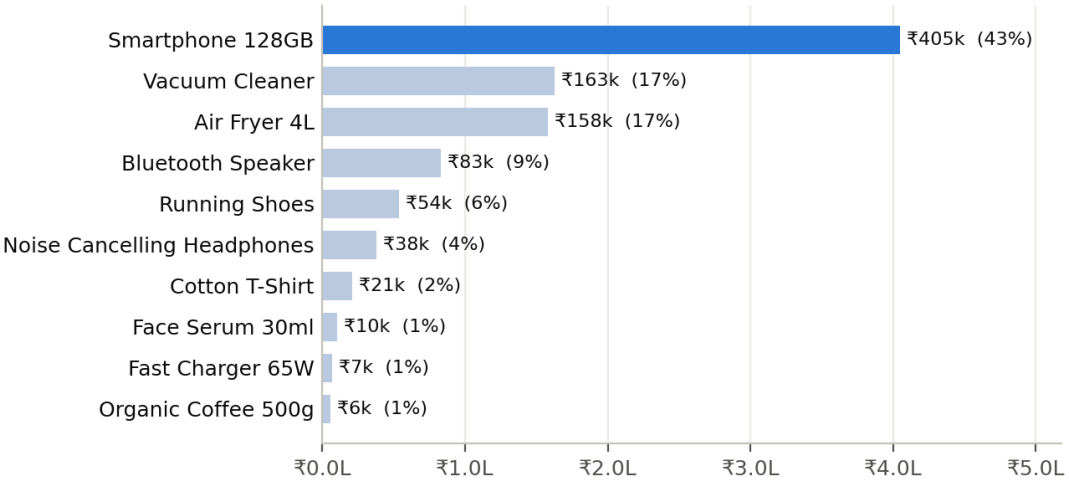


Figure 2: Net revenue by product (cancelled orders excluded).

3.2 The best sellers are the ones running out

11 of the 37 product and warehouse positions are below their reorder level. WH-WEST-02 is the worst, with four of them, including the Smartphone at 1 unit against a reorder level of 50 and the Air Fryer 4L at 3 units against 100. Those are the first and third highest revenue products. The Noise Cancelling Headphones are down to 2 units at WH-NORTH-01. A nightly stock report would flag the same rows, but only after a day of orders had already been routed to a warehouse that cannot ship them. The live table flags them as soon as the ERP row arrives on inventory-topic.

3.3 Payment failures are a customer problem, not a gateway problem

75 of the 100 payment events succeeded. The 16 failures were worth ₹2.28 lakh. Failure rates across gateways sit in a narrow band, Razorpay at 14.7 percent and Cashfree and PayU at 16.7 percent each, so switching providers would not fix much in this sample. The spread by payment method is wider. Wallet payments succeed 69 percent of the time and netbanking 84 percent. Half of all failures (8 of 16) are INSUFFICIENT_FUNDS, with BANK_TIMEOUT at 5 and AUTH_DECLINED at 3. When the shopper does not have the money in the wallet or account, a pay-later or EMI option at the retry step is more likely to save the sale than a gateway switch.

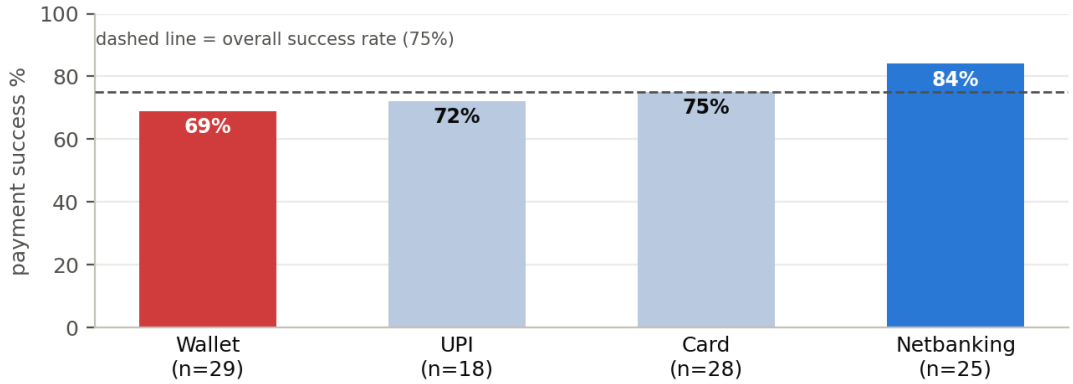


Figure 3: Payment success rate by method. Wallet sits well below the other methods.

Joining payments to orders shows the money at risk. 21 orders that are not cancelled have a payment in FAILED or PENDING state, worth ₹1.80 lakh in net order value. That is 19 percent of net revenue that the order table counts but the bank has not confirmed.

3.4 Carts are filled and then abandoned

The clickstream recorded 20 add-to-cart events but only 7 checkout starts and 4 purchase clicks. Two out of three carts never reach checkout, and there were 9 remove-from-cart events against those 20 adds. Running Shoes stand out, with 3 removals against 1 add. Once a shopper starts checkout, 4 of 7 go on to click purchase, so the leak is between cart and checkout, not inside the payment page. Guest sessions made up 14 percent of events, so forcing a login before checkout would put friction in front of a real share of traffic.

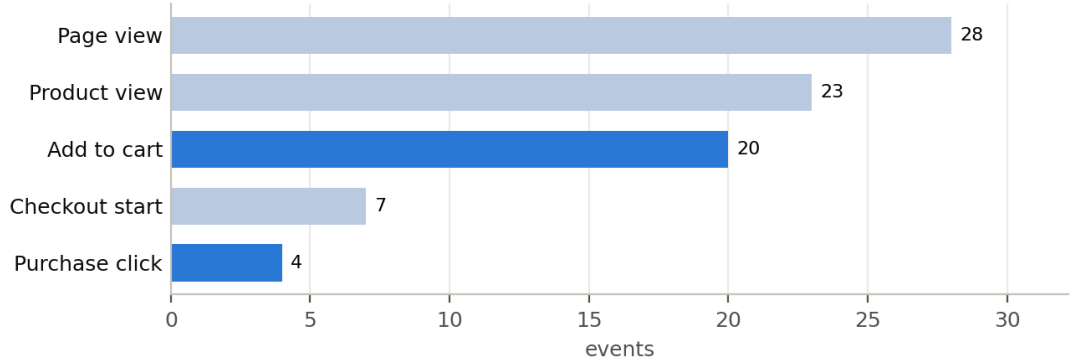


Figure 4: Clickstream funnel. The big drop sits between add to cart and checkout start.

3.5 COD orders cancel four times as often

15 orders used cash on delivery and 3 of them were cancelled, a 20 percent cancellation rate. Prepaid orders cancelled at 4.7 percent (4 of 85). COD cancellations cost ₹13,011 in order value here, plus a wasted delivery attempt each time in real operations.

3.6 Order count and order value point to different cities

Chennai placed the most orders (16), but Jaipur produced the most revenue, ₹1.78 lakh from 13 orders, at an average ticket of ₹13,705. Delhi placed 14 orders at an average of only ₹6,010. A city ranking by order count and one by revenue would send the marketing

budget to different places. For high-ticket categories like Mobiles and Home, Jaipur and Kolkata (average ₹13,458) look like better targets than their order volume suggests.

3.7 Discounts are not growing the basket

62 of 100 orders carried a discount, costing ₹76,802, or 7.2 percent of gross order value. Discounted orders averaged 2.05 units against 1.97 for full-price orders, which is effectively no difference. In this sample the discounts are being given away without changing how much people buy.

4. Recommended Actions

Dashboard signal	Trigger	Action	Owner
Low-stock alerts table	Shortfall on a top-3 revenue SKU	Raise a replenishment or inter-warehouse transfer for WH-WEST-02 (Smartphone, Air Fryer) today	Supply chain
Payment success tile	Below 70% for 10 minutes	If failures spread across all gateways, push pay-later or EMI at the retry step. If one gateway fails alone, reroute to the backup	Payments
Orders joined to failed payments	Open order with FAILED or PENDING payment	Send a payment retry link within the hour, before the order is auto-cancelled	CRM
Clickstream funnel	Add-to-cart to checkout below 40%	Trigger a cart reminder, and show delivery date and total cost on the cart page	Growth
Order status mix	COD share of cancellations rising	Offer a small prepaid discount, or require confirmation for COD orders above a value limit	Operations
Revenue by category and city	Weekly review	Move campaign spend toward Jaipur and Kolkata for Mobiles and Home. Test cutting the flat discount on low-ticket items	Marketing

5. Limitations and Learnings

100 records per source is enough to prove the pipeline works but too few to support pricing or procurement decisions on its own. A 5-point gap in payment success between two methods can come from three or four transactions. The numbers above should be read as the kind of signal the dashboard will surface at production volume, and the actions as the playbook that goes with each panel.

The most useful thing the consumer found was a data quality problem. When payments were joined to orders on order_id, none of the 100 payment amounts matched the order's net amount, and 80 of the 100 payments used a different method from the one on the order. The 15 COD orders also had an online payment record. In the sample data this happens because the generator draws payment amounts independently. In production it is exactly what a reconciliation check should raise, and it is why the dashboard reports order revenue and payment value separately instead of adding them together.

Two design choices mattered more than expected. Writing a stream_log row per message cost one extra insert, but it gave the dashboard a throughput panel that shows at a glance whether the pipeline is alive, which the business panels cannot do on their own

because a stalled stream simply looks like a quiet hour. Committing Kafka offsets manually after the database commit, combined with idempotent upserts, meant the consumer could be stopped and restarted mid-stream during testing without losing or duplicating a single order.